



ISC 互联网安全大会



360 互联网安全中心

基于深度学习的软件漏洞智能检测

邹德清 华中科技大学 教授

2018 ISC 互联网安全大会 中国·北京
Internet Security Conference 2018 Beijing·China
(原中国互联网安全大会)

目录

问题描述
主要贡献
系统设计
有效性评价
局限性与开放问题
总结

问题描述

背景

问题描述

研究现状

- 尽管在朝着安全编程的方向不断努力，安全漏洞问题仍然广泛存在



CVE中2010年登记了4,639个漏洞，2017年14,650个漏洞

- 目前有很多开源工具、商业产品及学术界研究，静态检测各种类型的漏洞

专家人工生成
漏洞规则

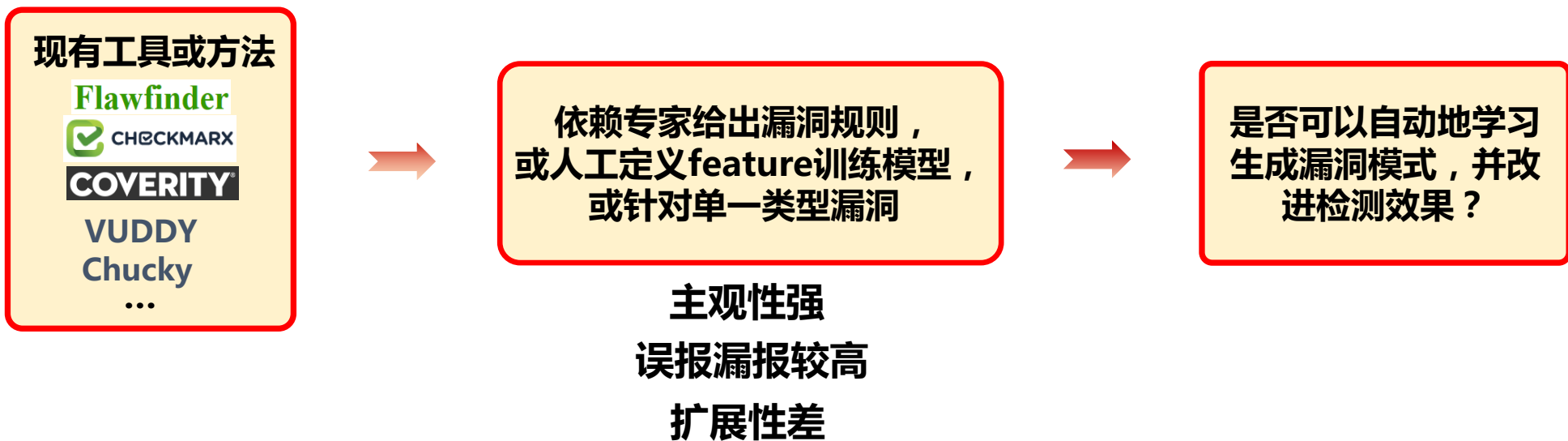
开源工具及商业产品，如Flawfinder, RATS, Checkmarx, Coverity, Fortify等，误报漏报较高

针对某类漏洞，半自
动生成漏洞模式

针对某类具体的漏洞，如格式化字符串漏洞，污点漏洞，信息泄露漏洞，缺少检查漏洞等

不区分漏洞类型，半
自动生成漏洞模式

采用机器学习技术，依赖专家定义features来刻画漏洞；由于粒度粗，无法定位漏洞具体位置



给定目标程序源码，如何在不需人类专家定义feature的前提下，以较低的误报和漏报，自动检测目标程序是否含有漏洞？如果有的话，给出漏洞代码的位置。

- 深度学习为解决上述问题提供了一种可能
- 深度学习在程序分析领域的应用

软件缺陷预测

代码克隆检测

API学习

恶意URL，
文件路径检测

二进制函数边界
识别

注册表键检测

- 深度学习应用于漏洞检测刚刚起步

存在的问题：只能检测某个函数是否含有漏洞，粒度太粗

CCS 2017- POSTER: vulnerability discovery with function representation learning from unlabeled projects

arXiv 2018- Automated vulnerability detection in source code using deep representation learning

arXiv 2018- Automated software vulnerability detection with machine learning

主要贡献

提出了第一个基于深度学习进行漏洞检测的系统化框架SySeVR，
构建了基于深度学习的细粒度漏洞检测系统VulDeePecker，
将人类专家从繁琐、主观的手工定义feature中解放出来，并显著
降低误报漏报

系统设计

将深度学习用于漏洞检测的指导原则

定义SYVC和SEVC

系统化结构框架SYSEVR

基于深度学习的漏洞检测系统VULDEEPECKER

将深度学习用于漏洞检测的指导原则

➤ 如何表征程序？

指导原则1: 程序需首先转化为保留程序语义（如数据流、控制流）的中间表示，然后转化为向量表征，作为神经网络的输入

➤ 如何选择代码粒度？

指导原则2: 程序需要以比文件、函数等更细粒度的单位表征代码，从而可以定位漏洞

➤ 如何选择神经网络？

指导原则3: 由于代码中是否含有漏洞依赖于上下文，因此能够处理上下文的神经网络更适用于漏洞检测

➤ SyVC (Syntax-based Vulnerability Candidates)

SyVC是一个代码元素，由代码中一个或多个token组成；该代码元素满足已知漏洞的某个**基本语法特性**，可能是漏洞，也可能不是漏洞

基本语法特性：可通过人工分析现有的漏洞规则进行初步归类得到；仅是漏洞检测的出发点

库/API函数调用

数组使用

指针使用

算术表达式

➤ SeVC (Semantics-based Vulnerability Candidates)

SeVC是SyVC的扩展，由与SyVC语义相关（如控制依赖和数据依赖）的语句组成

SyVC和SeVC举例

```
1 void printLine(const char * line)
2 {
3     if(line != NULL)
4         printf("%s\n", line);
5 }
6
7 void func()
8 {
9     char *data;
10    char dataBuffer[100];
11    char source[100];
12    ...
13    memset(dataBuffer, 'A', 99);
14    dataBuffer[99] = '\0';
15    ...
16    while(1)
17    {
18        data = dataBuffer - 8;
19        break;
20    }
21    ...
22    memset(source, 'C', 99);
23    source[99] = '\0';
24    memmove(data, source,
25            100*sizeof(char));
26    data[99] = '\0';
27    printLine(data);
28 }
```

源代码

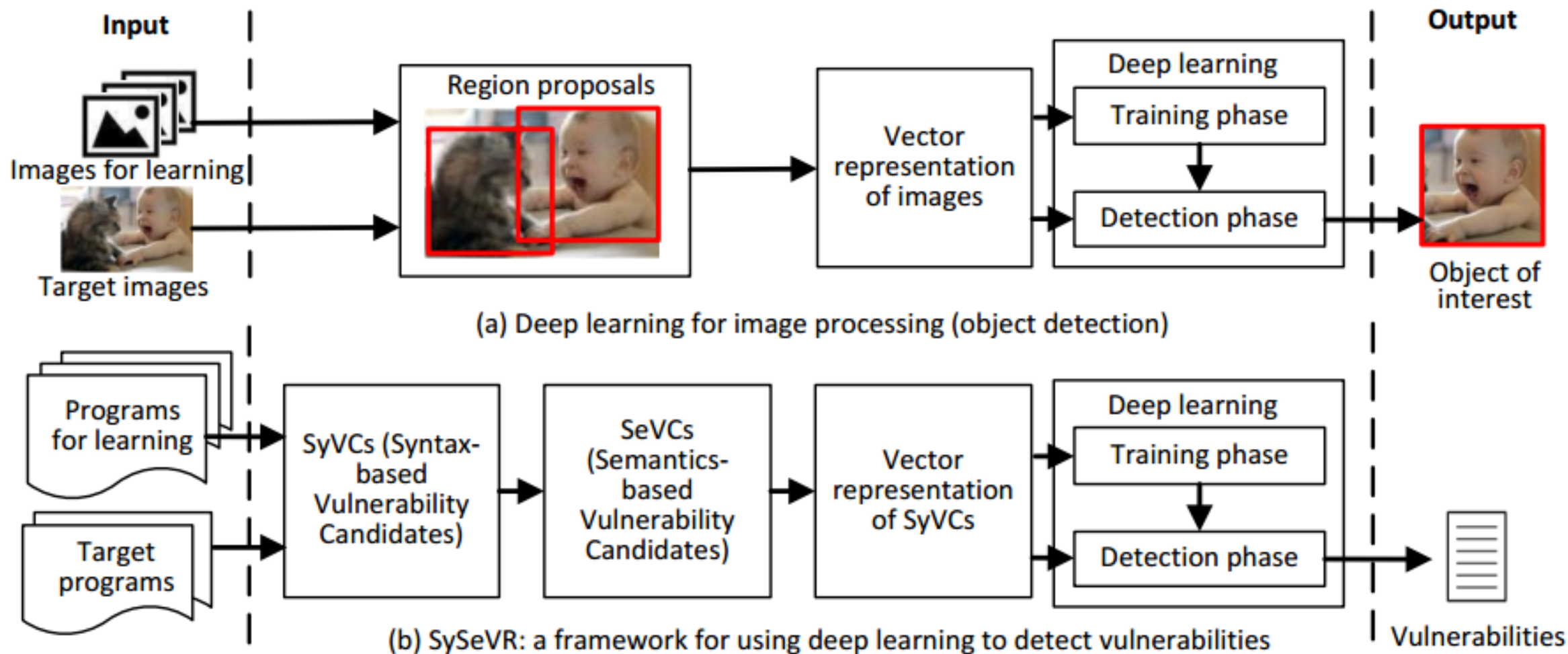
```
1 void printLine(const char * line)
2 {
3     if(line != NULL)
4         printf("%s\n", line);
5 }
6
7 void func()
8 {
9     char *data;
10    char dataBuffer[100];
11    char source[100];
12    ...
13    memset(dataBuffer, 'A', 99);
14    dataBuffer[99] = '\0';
15    ...
16    while(1)
17    {
18        data = dataBuffer - 8;
19        break;
20    }
21    ...
22    memset(source, 'C', 99);
23    source[99] = '\0';
24    memmove(data, source,
25            100*sizeof(char));
26    data[99] = '\0';
27    printLine(data);
28 }
```

SyVC (红框标出)

```
7 void func()
9 char *data;
10 char dataBuffer[100];
11 char source[100];
13 memset(dataBuffer, 'A', 99);
14 dataBuffer[99] = '\0';
16 while (1)
18 data = dataBuffer - 8;
22 memset(source, 'C', 99);
23 source[99] = '\0';
24 memmove (data, source,
25         100*sizeof(char));
26 data[99] = '\0';
27 printLine(data);
1 void printLine(const char * line)
3 if(line != NULL)
4 printf("%s\n", line);
```

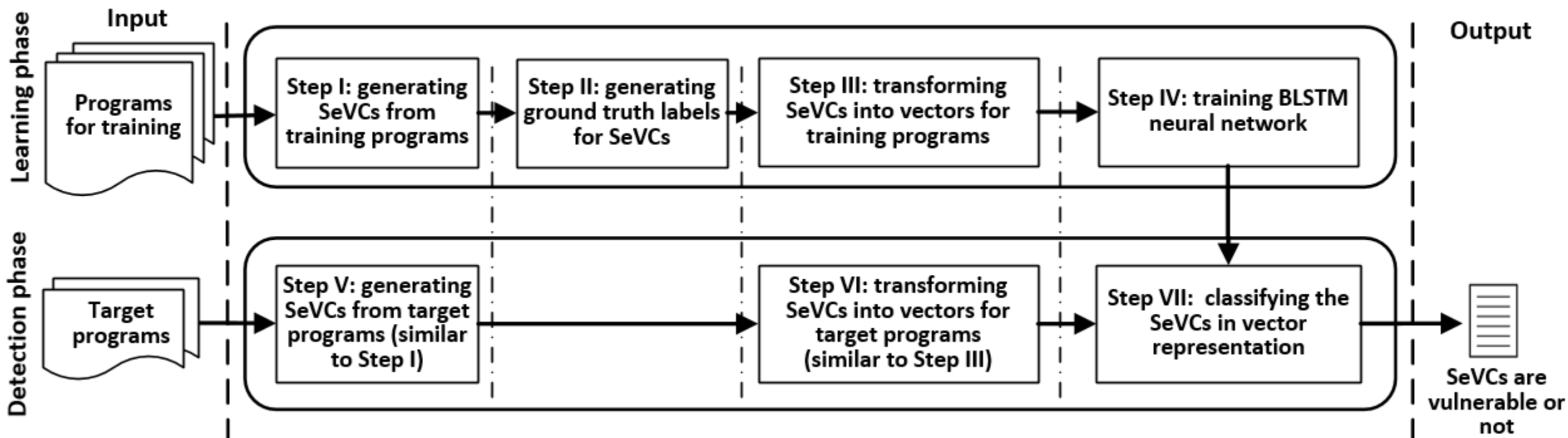
第25行SyVC “data”
对应的SeVC

系统化结构框架SySeVR



实例：基于深度学习的漏洞检测系统VulDeePecker

- 以库/API函数调用作为SyVC



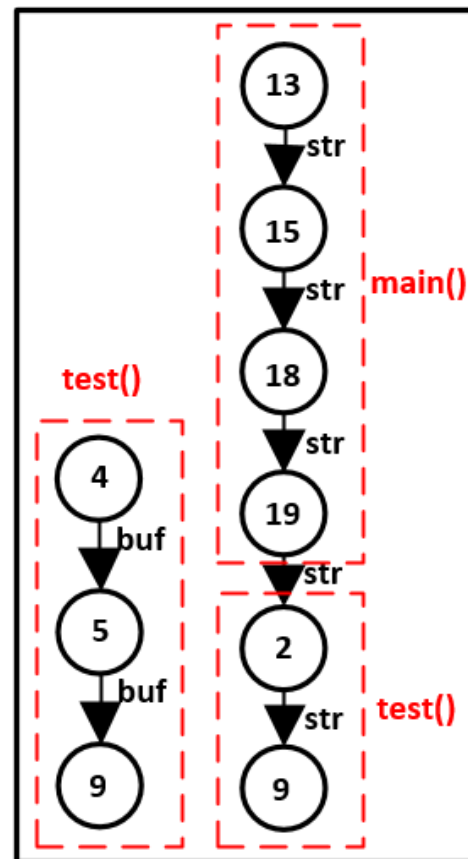
基于深度学习的漏洞检测系统VulDeePecker

➤ Step I : 生成SeVC

```
1 void
2 test(char *str)
3 {
4     int MAXSIZE=40;
5     char buf[MAXSIZE];
6
7     if(!buf)
8         return;
9     strcpy(buf, str); /*string copy*/
10 }
11
12 int
13 main(int argc, char **argv)
14 {
15     char *userstr;
16
17     if(argc > 1) {
18         userstr = argv[1];
19         test(userstr);
20     }
21     return 0;
22 }
```



9 strcpy(buf, str)



A SeVC corresponding to strcpy()

```
13 main(int argc, char **argv)
15 char *userstr;
18 userstr = argv[1];
19 test(userstr);
2 test(char *str)
4 int MAXSIZE=40;
5 char buf[MAXSIZE];
9 strcpy(buf, str); /*string copy*/
```

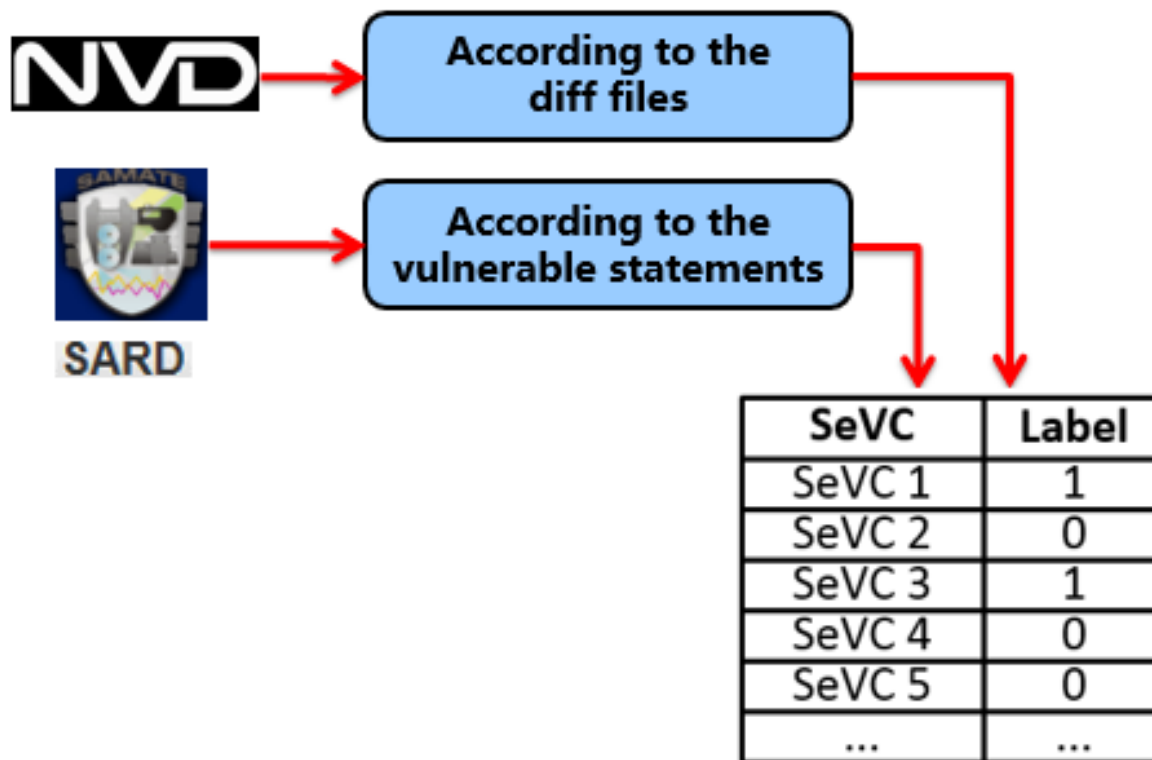
(3) Assemble slices into SeVCs

(1) Extract library/API function calls

(2) Generate slices for arguments of library/API function calls

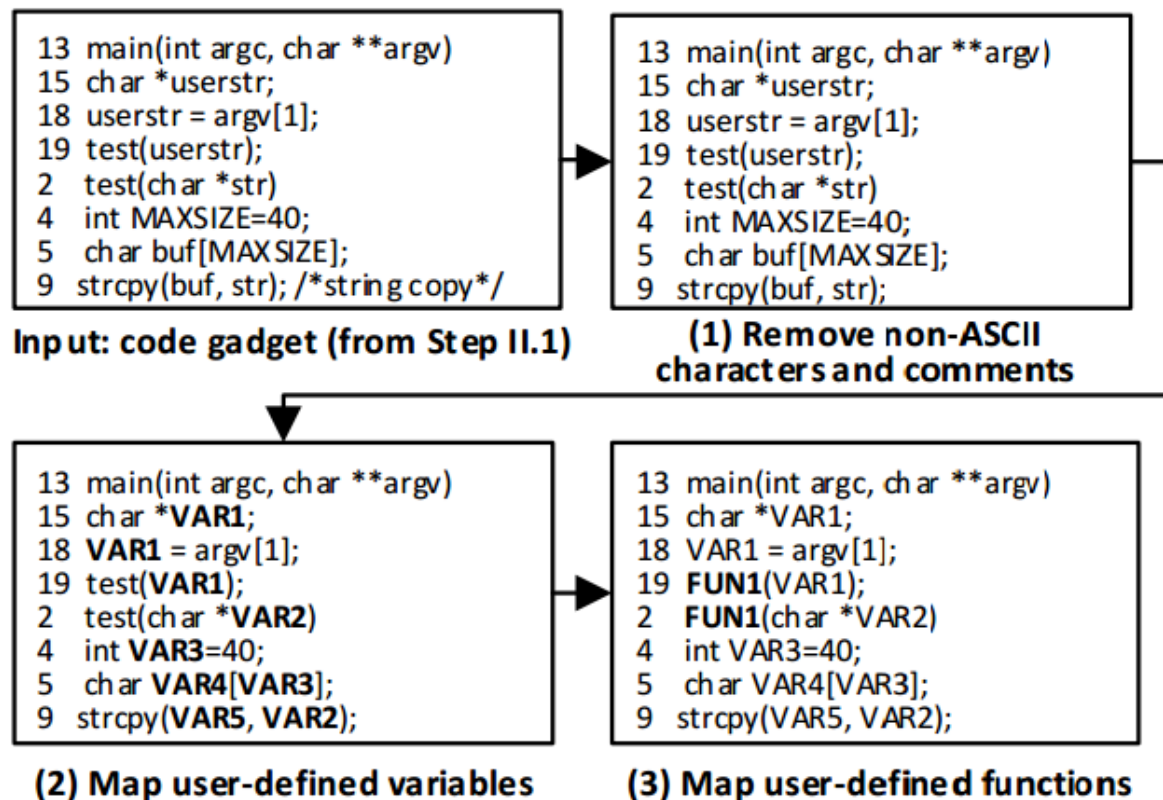
➤ Step II : 为SeVC生成ground truth标签

每个SeVC标记为 “1” （有漏洞）或 “0” （无漏洞）

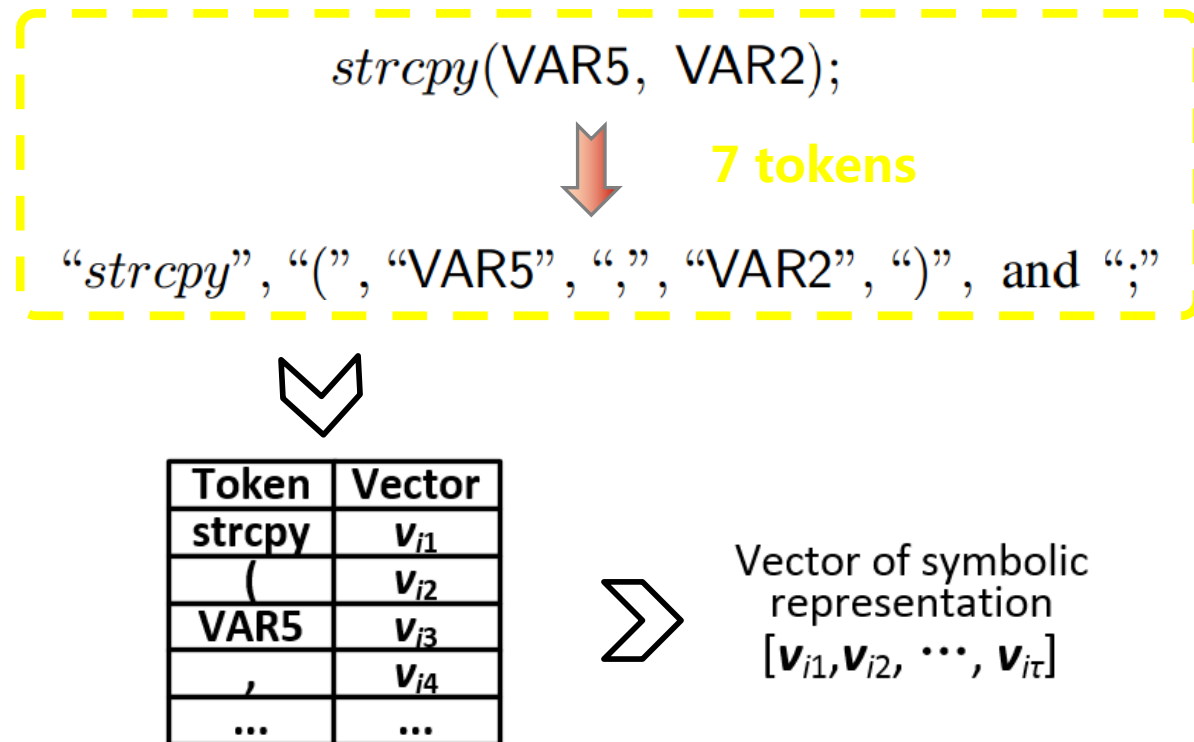


➤ Step III : 将SeVC转换为向量

◆ 将SeVC转换为符号表征

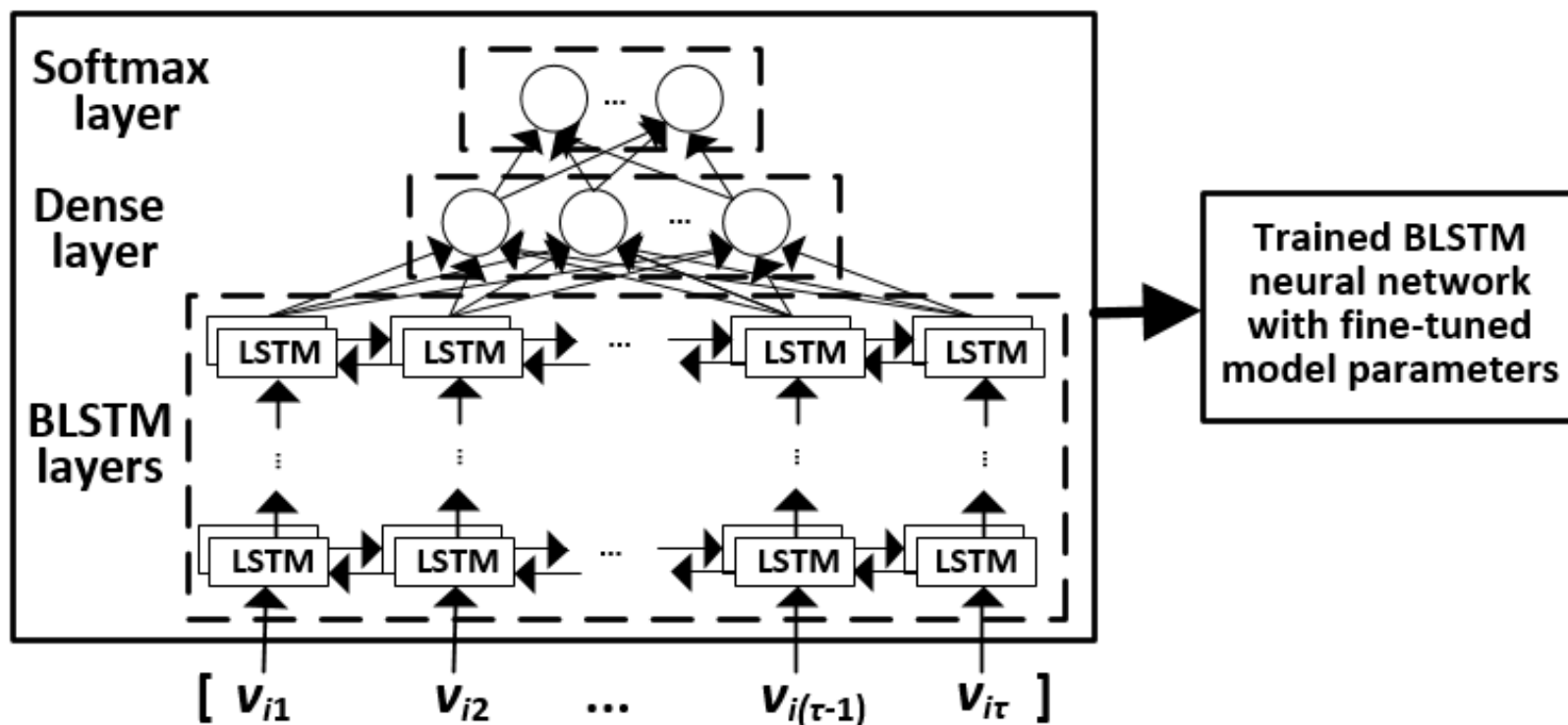


◆ 将符号表征编码为向量

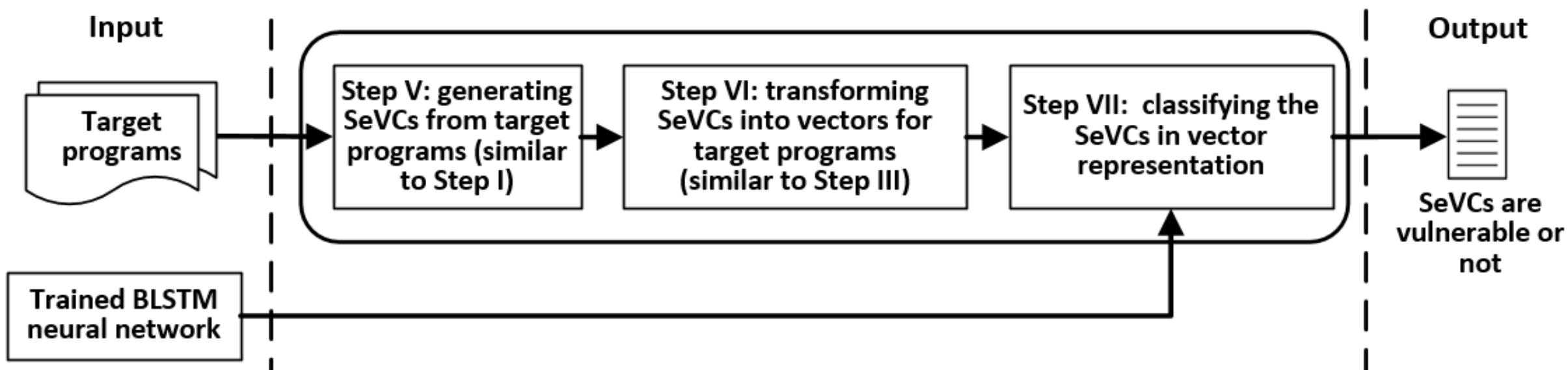


基于深度学习的漏洞检测系统VulDeePecker

- Step IV : 采用标准的BLSTM模型训练



➤ Step V-VII : 检测阶段



有效性评价

数据集

研究问题

真实软件中的检测效果

- 数据来源：NVD中公布的19 个C/C++开源软件的漏洞，SARD中C/C++ test cases
 - ◆ 针对缓冲区溢出漏洞（CWE-119），采集了520 个开源软件程序文件和8,122 个test cases
 - ◆ 针对资源管理漏洞（CWE-399），采集了320个开源软件程序文件和 1,729 个test cases
 - ◆ 80% 作为训练程序，20%作为目标程序

- 数据集含61,638个SeVC

数据集开源：<https://github.com/CGCL-codes/VulDeePecker>

BE: Buffer error vulnerabilities

RM: Resource management vulnerabilities

HY: Hybrid of the above two types of vulnerabilities

ALL: All library/API function calls

SEL: Manually selected library/API function calls

Dataset	#SeVCs	#Vulnerable SeVCs	#Not vulnerable SeVCs
BE-ALL	39,753	10,440	29,313
RM-ALL	21,885	7,285	14,600
HY-ALL	61,638	17,725	43,913
BE-SEL	26,720	8,119	18,601
RM-SEL	16,198	6,573	9,625
HY-SEL	42,918	14,692	28,226

- RQ1 : VulDeePecker能同时处理多类漏洞吗？

Insight: VulDeePecker可以应用到多类漏洞，其有效性与安全相关库/API函数的个数有关

Dataset	FPR(%)	FNR(%)	TPR(%)	P(%)	F1(%)
BE-ALL	2.9	18.0	82.0	91.7	86.6
RM-ALL	2.8	4.7	95.3	94.6	95.0
HY-ALL	5.1	16.1	83.9	86.9	85.4

RM: 16 function calls related to vulnerabilities

BE: 124 function calls related to vulnerabilities

- RQ2 : 人类经验 (非定义特征) 能够改进 VulDeePecker 的有效性吗 ?

Insight: 人类经验可用于选择和安全有关的库/API函数 , 能够改进 VulDeePecker 的有效性

Dataset	FPR(%)	FNR(%)	TPR(%)	P(%)	F1(%)
HY-ALL	5.1	16.1	83.9	86.9	85.4
HY-SEL	4.9	6.1	93.9	91.9	92.9

➤ RQ3 : VulDeePecker 与其他静态漏洞检测方法相比有效性如何？

Insight: VulDeePecker 通过采用数据流信息比人工定义规则的静态分析工具（开源工具和商业工具）更有效

System	Dataset	FPR (%)	FNR (%)	TPR (%)	P (%)	F1 (%)
VulDeePecker vs. Other pattern-based vulnerability detection systems						
Flawfinder	BE-SEL	44.7	69.0	31.0	25.0	27.7
RATS	BE-SEL	42.2	78.9	21.1	19.4	20.2
Checkmarx	BE-SEL	43.1	41.1	58.9	39.6	47.3
VulDeePecker	BE-SEL	5.7	7.0	93.0	88.1	90.5
VulDeePecker vs. Code similarity-based vulnerability detection systems						
VUDDY	BE-SEL-NVD	0	95.1	4.9	100	9.3
VulPecker	BE-SEL-NVD	1.9	89.8	10.2	84.3	18.2
VulDeePecker	BE-SEL-NVD	22.9	16.9	83.1	78.6	80.8
VUDDY	BE-SEL-SARD	N/C	N/C	N/C	N/C	N/C
VulPecker	BE-SEL-SARD	N/C	N/C	N/C	N/C	N/C
VulDeePecker	BE-SEL-SARD	3.4	5.1	94.9	92.0	93.4

- RQ3 : VulDeePecker 与其他静态漏洞检测方法相比有效性如何？

Insight: VulDeePecker比基于代码相似性的漏洞检测方法具有更低的漏报

System	Dataset	FPR (%)	FNR (%)	TPR (%)	P (%)	F1 (%)
VulDeePecker vs. Other pattern-based vulnerability detection systems						
Flawfinder	BE-SEL	44.7	69.0	31.0	25.0	27.7
RATS	BE-SEL	42.2	78.9	21.1	19.4	20.2
Checkmarx	BE-SEL	43.1	41.1	58.9	39.6	47.3
VulDeePecker	BE-SEL	5.7	7.0	93.0	88.1	90.5
VulDeePecker vs. Code similarity-based vulnerability detection systems						
VUDDY	BE-SEL-NVD	0	95.1	4.9	100	9.3
VulPecker	BE-SEL-NVD	1.9	89.8	10.2	84.3	18.2
VulDeePecker	BE-SEL-NVD	22.9	16.9	83.1	78.6	80.8
VUDDY	BE-SEL-SARD	N/C	N/C	N/C	N/C	N/C
VulPecker	BE-SEL-SARD	N/C	N/C	N/C	N/C	N/C
VulDeePecker	BE-SEL-SARD	3.4	5.1	94.9	92.0	93.4

- 针对4个目标软件：Libav, Seamonkey, Thunderbird, and Xen
- VulDeePecker：检测出4个NVD中未公布的漏洞，但在软件后续版本中进行了默默修补

Target product	CVE ID	Vulnerable product published in the NVD	Vulnerability publish time	Vulnerable file in target product	Library/API function call	1st patched version of target product
Libav 10.1	CVE-2013-0851	FFmpeg	12/07/2013	libavcodec/eamad.c	memset	Libav 10.3
Seamonkey 2.31	CVE-2015-4517	Firefox	09/24/2015	.../dom/system/gonk/NetworkUtils.cpp	snprintf	Seamonkey 2.38
	CVE-2015-4513	Firefox	11/05/2015	.../network/protocol/http/Http2Stream.cpp	memset	Seamonkey 2.39
Xen 4.6.0	CVE-2016-9104	Qemu	12/09/2016	tools/qemu-xen/hw/9pfs/virtio-9p.c	memcpy	Xen 4.9.0

- SySeVR（支持4类SyVC）：检测出15个NVD中未公布的漏洞，其中7个未知漏洞，8个漏洞在软件后续版本中进行了默默修补

局限性与开放问题

- 局限于C/C++源代码漏洞检测
- 目前SyVC和SeVC的实现中容纳的语法和语义信息有限
- 以SeVC为粒度检测漏洞，无法准确定位到漏洞行
- 数据集标签的生成过程中采用了启发式方法
- 需要更多的研究来对各深度学习模型的检测效果进行解释

总结

- 提出了SySeVR，第一个基于深度学习进行漏洞检测的系统化框架，构建了基于深度学习的细粒度漏洞检测系统VulDeePecker，将人类专家从繁琐、主观的手工定义feature解放出来，并降低误报漏报
- 探讨了将深度学习用于漏洞检测的指导原则
- 构建了第一个基于深度学习进行漏洞检测的数据集
<https://github.com/CGCL-codes/VulDeePecker>
- 系统地评价了VulDeePecker的有效性



ISC 互联网安全大会



360 互联网安全中心

谢谢！

2018 ISC 互联网安全大会 中国·北京
Internet Security Conference 2018 Beijing·China
(原中国互联网安全大会)